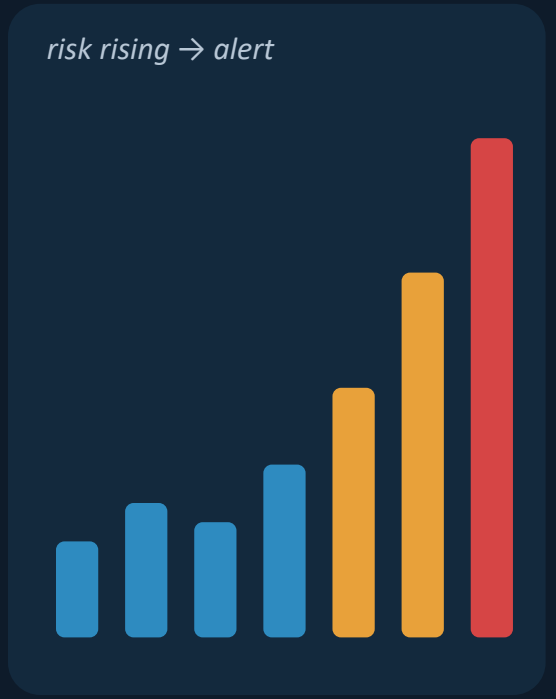
Review 2

OS Fault-Prediction Daemon

Predicts system faults before they happen, and explains them in plain English.

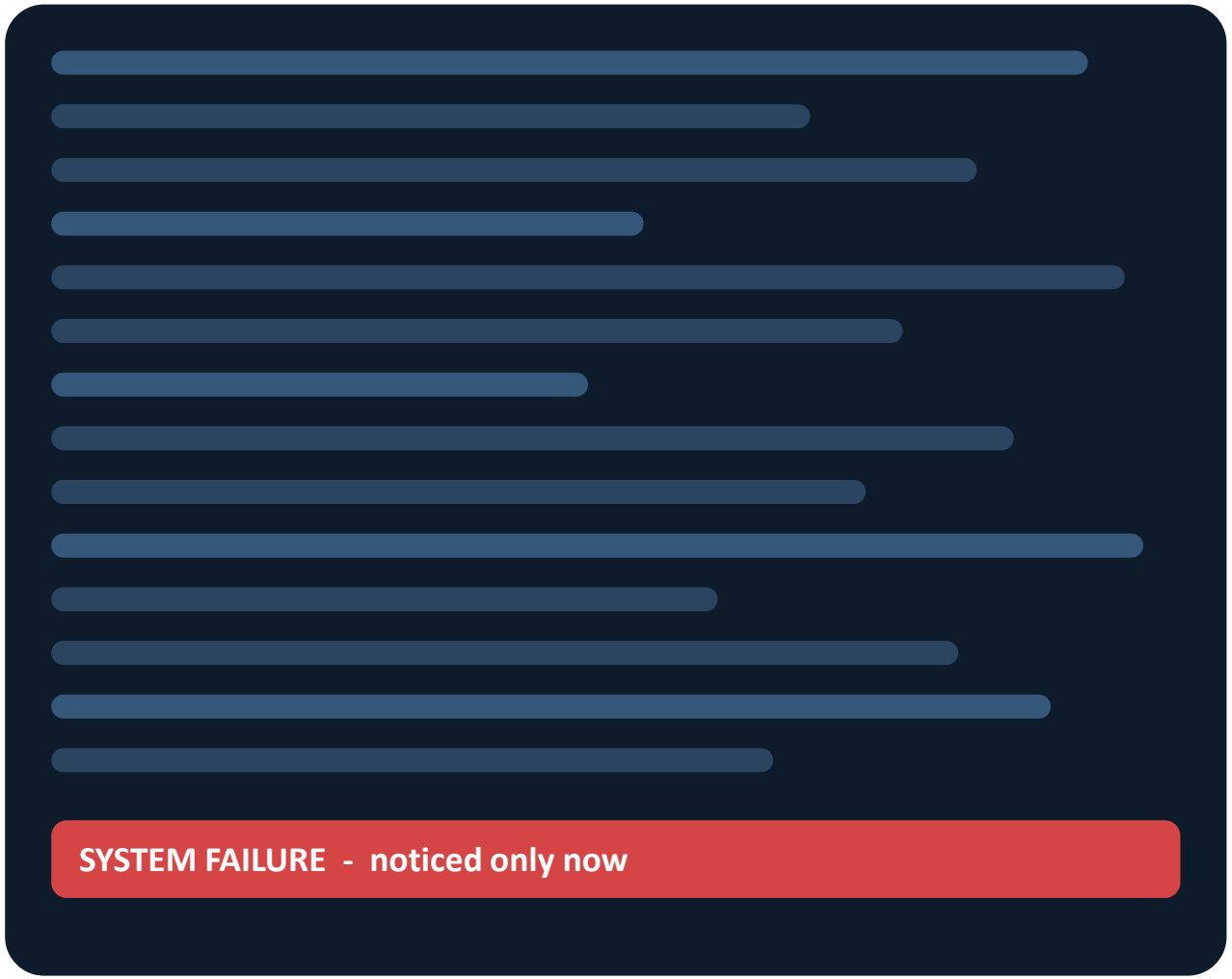


Ritvik Battula (24BAI0156)

Shaurya Kumar (24BCE0876)

VIT Vellore

The problem



4,713,493

log lines in one supercomputer system log (BGL)

585

distinct message types in the training period alone

After

the fault: that is when operators usually find out

Meet the product

Watch

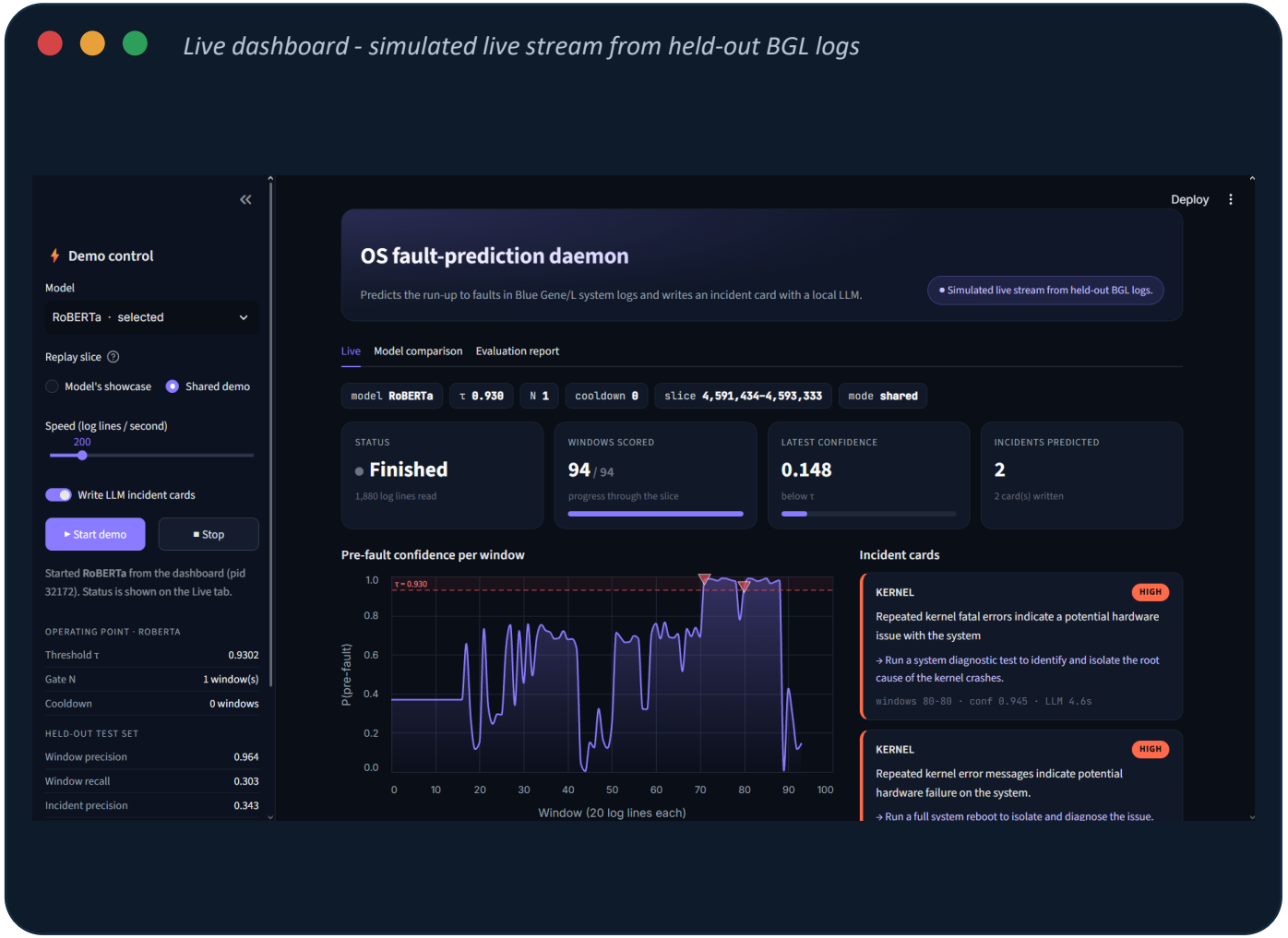
reads the system log as a live stream

Predict

spots the run-up to a fault early

Explain

writes a plain-English incident card



Live demonstration

1

Replay

held-out log lines stream in, 20 lines per window

2

Confidence climbs

the model scores every window; watch it cross the threshold

3

Incident fires

the alert gate turns a run of flagged windows into one incident

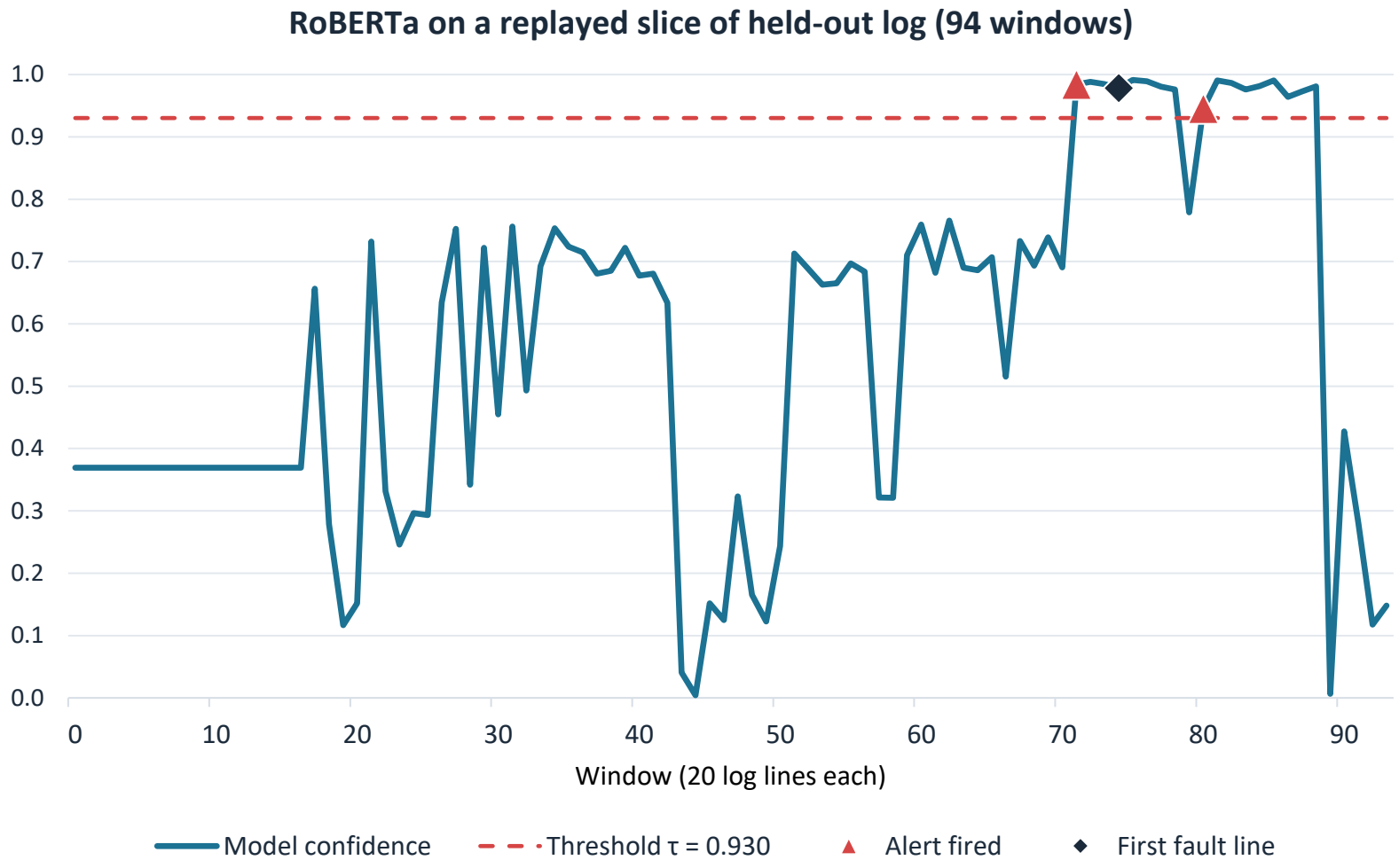
4

Card appears

the local LLM writes summary, component, severity, action

Simulated live stream from held-out BGL logs.

One fault, start to finish



59 lines

warning before the first fault line (9 s of log time)

2 incidents

1 before the fault, 1 once it had started, 0 false alarms on this slice

System architecture

Dashboard
Streamlit

- live log feed
- confidence chart
- incident cards
- demo launcher

Daemon
Python service

- stream loop
- windowing
- alert gate
- incident builder

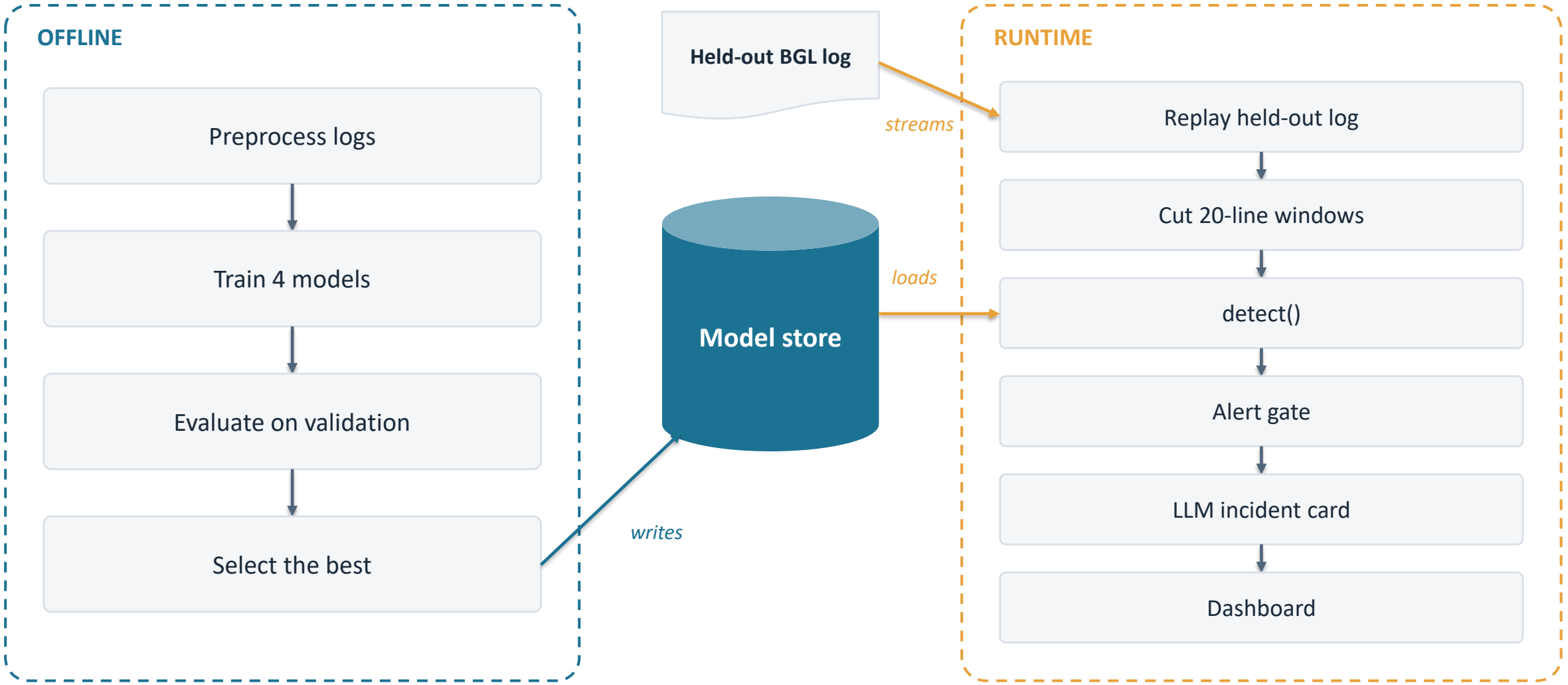
Models
PyTorch + Ollama

- RoBERTa detect()
- Llama 3.2 3B incident writer

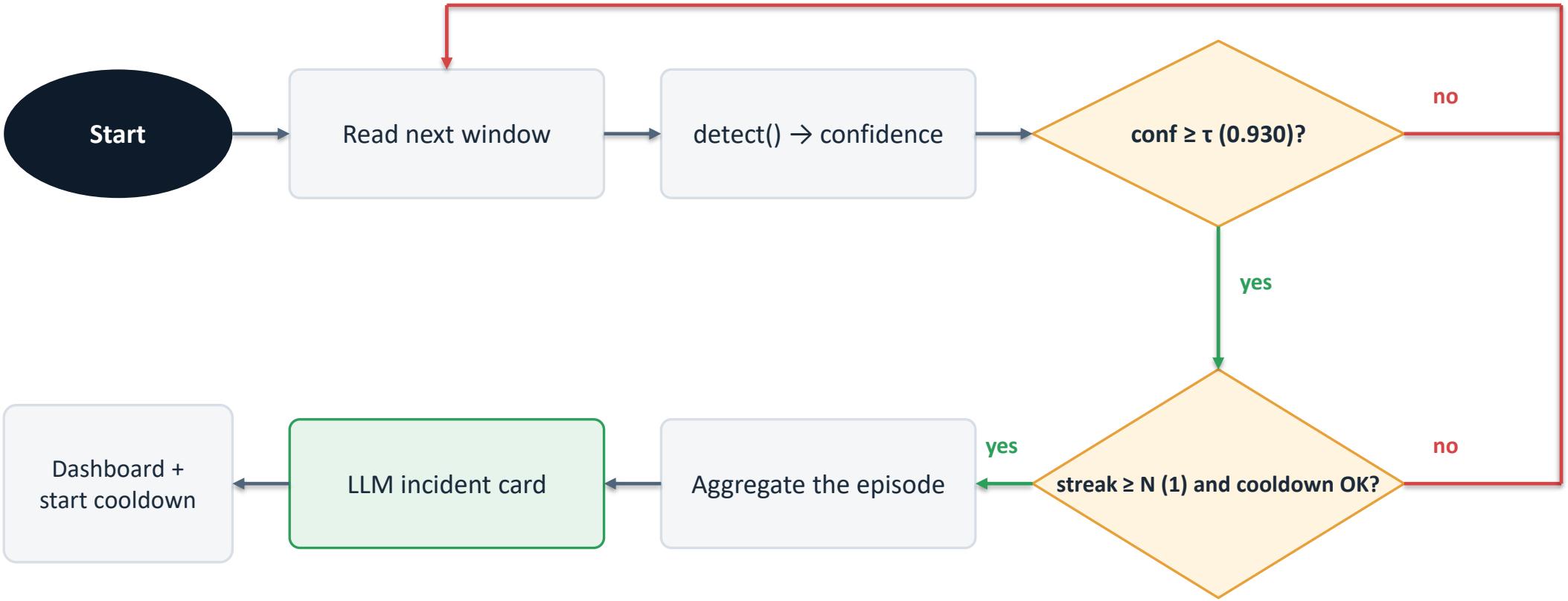
Data
BGL + Drain3

- BGL system log
- Drain3 templates
- 20-line windows
- saved models

System design



How it decides



One fault episode = one alert: a run of flagged windows becomes a single incident.

Dataset pipeline

4,713,493
raw log lines

585
Drain3 templates (from training lines)

112,331
labelled 20-line windows

2.83%
pre-fault windows in held-out test

How a window is labelled

20-line window | **look-ahead: next 100 lines** | **fault**

Pre-fault if a fault line follows within 100 lines.
Windows that already contain a fault line are excluded.

Split	Windows	Pre-fault	Pre-fault rate
Train	47,311	4,301	9.09%
Validation	20,896	1,087	5.20%
Test (held out)	44,124	1,250	2.83%

Chronological split 70 / 10 / 20. Train negatives sampled 10:1; validation and test keep the real rate.

Module identification

1. **Data preprocessing** — Operational
2. **Model training & selection** — Operational
3. **Daemon / stream loop** — Operational
4. **Alert gate (τ / N / cooldown)** — Operational
5. **Incident reporting (LLM)** — Operational
6. **Dashboard** — Operational

Planned upgrades

- Detector explainability
- Drift monitor
- Incident correlation
- Live journalctl monitoring

Operational · Pending · Planned

Module completion

Operational (7)

- Data preprocessing
- Model training & selection
- Daemon / stream loop
- Alert gate (τ / N / cooldown)
- Incident reporting (LLM)
- Dashboard
- 4-model comparison with the best deployed

Pending (0)

- None: every core module runs end to end on the held-out stream

Planned (4)

- Detector explainability
- Drift monitor
- Incident correlation
- Live journalctl monitoring

Choosing the brain: 4 models tested

Model	Window precision	Window recall	Window F1	Test PR-AUC	Val PR-AUC	ms / window
RoBERTa ✓ Selected	**96.4%**	**30.3%**	**0.461**	**0.473**	**0.259**	**14.0**
DistilBERT	94.6%	17.0%	0.288	0.378	0.240	8.5
ELECTRA	95.4%	15.0%	0.259	0.431	0.217	14.3
Gradient-boosted trees	100.0%	1.2%	0.024	0.363	0.213	1.4

How to read this

Window-level metrics on unseen test data, each model at its own threshold (lowest threshold reaching 80% precision on validation). Selection metric: validation PR-AUC.

How each model performs

RoBERTa ✓ Selected

	Predicted normal	Predicted pre-fault
Actually normal	42,860	14 false alarms
Actually pre-fault	871 missed	379 caught

DistilBERT

	Predicted normal	Predicted pre-fault
Actually normal	42,862	12 false alarms
Actually pre-fault	1,038 missed	212 caught

ELECTRA

	Predicted normal	Predicted pre-fault
Actually normal	42,865	9 false alarms
Actually pre-fault	1,063 missed	187 caught

Gradient-boosted trees

	Predicted normal	Predicted pre-fault
Actually normal	42,874	0 false alarms
Actually pre-fault	1,235 missed	15 caught

False positives = false alarms; each model's threshold was set to keep them low. Window-level counts on held-out test data.

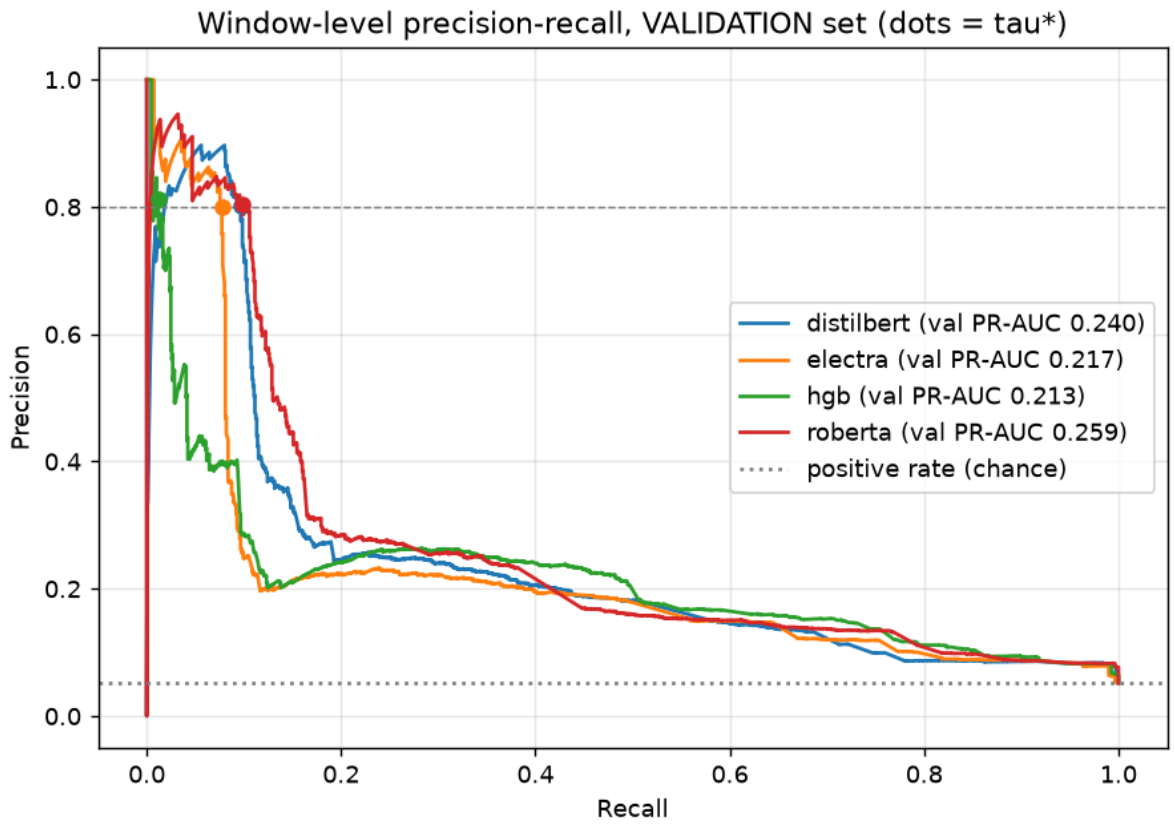
Alerts that matter: incident-level

Model	Gate (N, cooldown)	Alerts fired	Before the fault	During a fault	False alarms	Incident precision	Faults warned
RoBERTa ✓ Selected	**1, 0**	**105**	**36**	**62**	**7**	**34.3%**	**35 / 218**
DistilBERT	2, 30	52	30	17	5	57.7%	30 / 218
ELECTRA	1, 0	115	52	58	5	45.2%	51 / 218
Gradient-boosted trees	1, 5	17	12	5	0	70.6%	12 / 218

Incident-level, held-out test data

One incident = one alert to an operator. Incident precision counts only alerts that came before a fault started (within 100 lines); alerts during a fault are not credited.

Why we chose RoBERTa



Best at ranking risk

highest PR-AUC of the 4: validation 0.259, test 0.473

Fast enough for live use

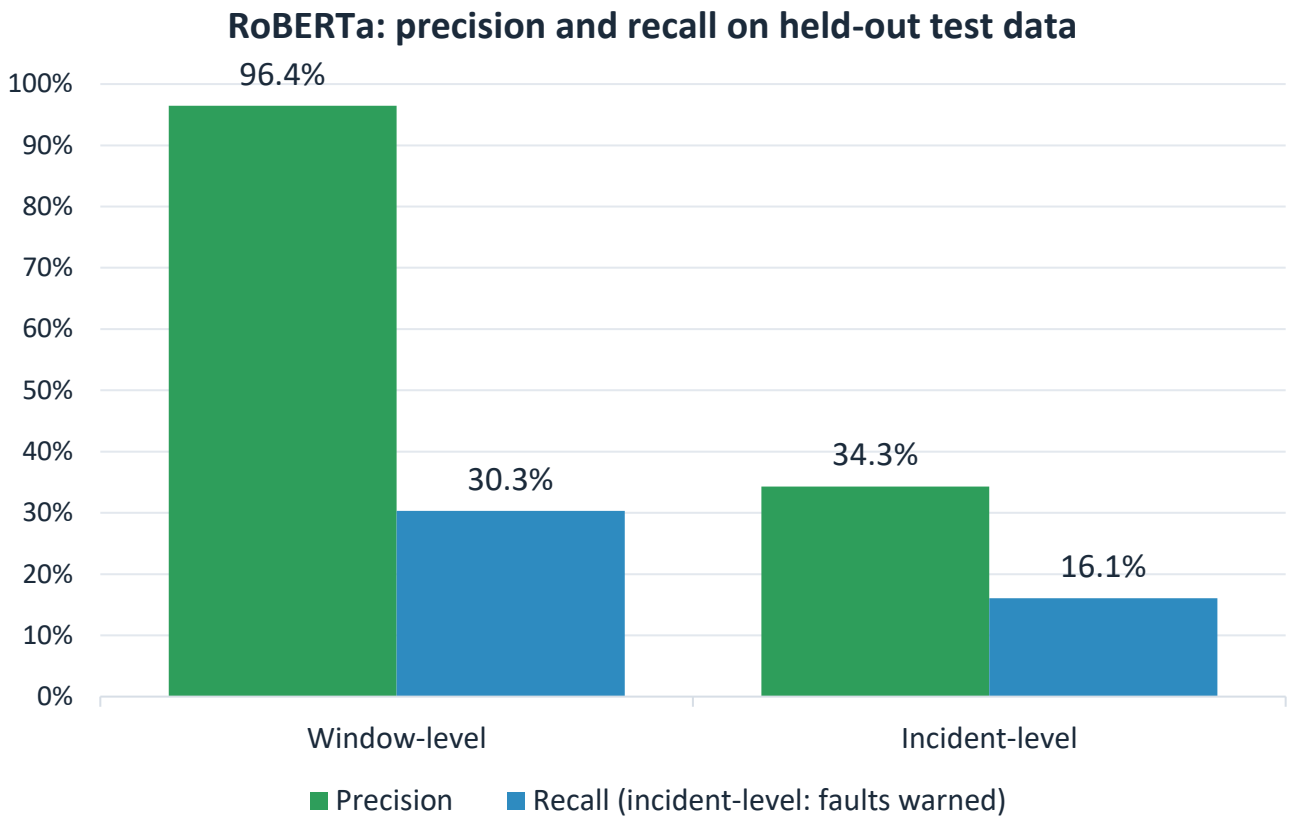
14.0 ms to score a 20-line window

Runs locally

3.7 GB peak VRAM while training on a 6 GB laptop GPU; no cloud needed

Picked on the validation set, confirmed on unseen test data.

Built for precision



34%

of alerts arrive before a real fault starts (incident-level)

93%

of alerts coincide with a real fault, before or during it (incident-level)

Whole windows kept: 1.1% truncated at 512 tokens

Ambiguity buffer: 200 lines

Tuned gate: τ 0.930, N 1, cooldown 0

What drives precision

Configuration	Val PR-AUC	Window precision	Window recall	Train time
RoBERTa, all design choices (full training)	**0.259**	**96.4%**	**30.3%**	**48 min**
All choices on, 1-epoch retrain (reference)	0.247	95.8%	16.2%	15 min
Without whole-window text rendering	0.196	94.7%	1.4%	8 min
Without the ambiguity buffer	0.217	97.7%	27.0%	15 min
Without overlapping training windows	0.219	95.7%	23.0%	8 min
Without negative downsampling	0.262	95.8%	25.8%	84 min

Window-level precision and recall on held-out test data.

Biggest lever

Whole-window text rendering

switching it off drops validation PR-AUC by 0.050

How it was tested

Each choice switched off in turn and the model retrained for one epoch; compare with the 1-epoch reference row.

Tuning the alert gate

Threshold τ

0.930

lowest score reaching 80% window precision on validation

Streak N

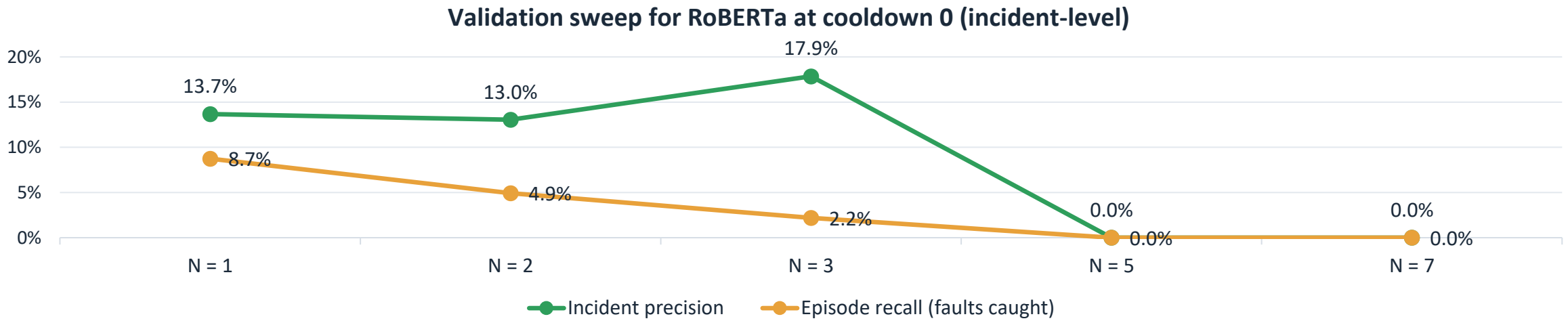
1 window

best balance of incident precision and faults caught on validation

Cooldown

0 windows

flagged runs already merge into one incident



Tuned on validation, never test.

What the operator gets

34%
incident precision: alerts that came before a fault

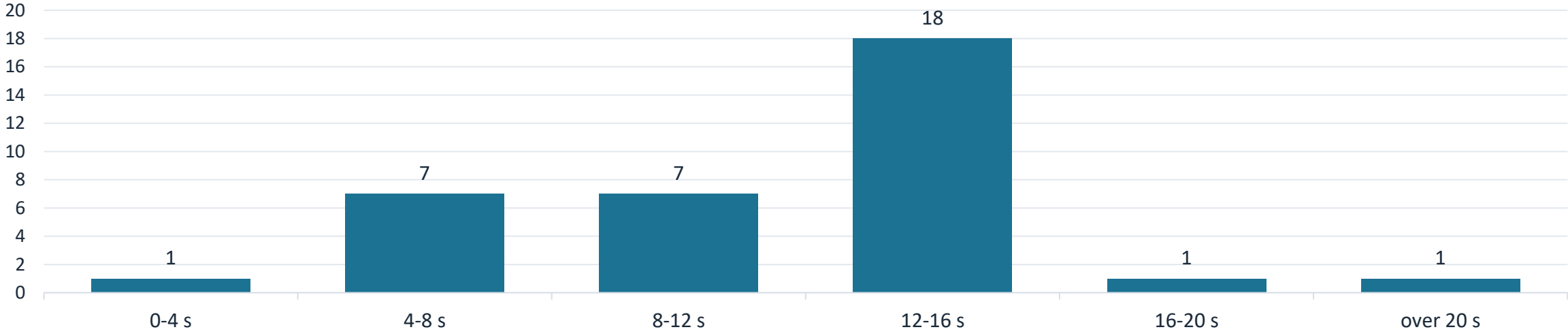
16%
of faults warned in advance (35 of 218)

12 s
median warning before the fault's first line

2.5 s
to write the incident card (llama3.2:3b, local)

14 ms
to score a 20-line window

How early the warning came: 35 faults warned in advance (held-out test)



Screenshots of all outputs

Preprocessing: templates, windows, splits

```
[parse] 4,713,493 well-formed lines (5s); mining templates on the first 3,299,445
[mine] 1,000,000 lines, 301 templates (16s)
[mine] 2,000,000 lines, 383 templates (34s)
[mine] 3,000,000 lines, 550 templates (42s)
[mine] 585 templates saved to drain3_state.bin
[encode] 1,000,000 lines (64s)
[encode] 2,000,000 lines (71s)
[encode] 3,000,000 lines (78s)
[encode] 4,000,000 lines (90s)
[split] no line overlap between train / val / test windows: OK
[train] 300,744 windows (4,301 pos) before filters; buffer dropped 3,940 negatives, downsampling dropped 249,493
[split] train   47,311 windows   4,301 positive  (9.09%)
[split] val     20,896 windows   1,087 positive  (5.20%)
[split] test    44,124 windows   1,250 positive  (2.83%)
[lines] train unmatched by a saved template: 0.00%
[lines] val   unmatched by a saved template: 9.52%
[lines] test  unmatched by a saved template: 45.91%
[render] example: KERNEL INFO generating * x20
[tokens] distilbert p50=13 p90=367 p95=433 p99=577 max=790  ->  max_length=512, 1.74% truncated
[tokens] roberta    p50=15 p90=354 p95=423 p99=526 max=726  ->  max_length=512, 1.12% truncated
```

Model comparison: the four models

1. The models

Model	Family	Input	Size	Training	Train (min)	Peak VRAM (MB)	Inference (ms/window)
distilbert	distilbert-base-uncased	rendered template text (RLE), max_length 512	67M parameters	best epoch 1 of 2 (early stop)	24.2	4220	8.5
electra	google/electra-base-discriminator	rendered template text (RLE), max_length 512	109M parameters	best epoch 1 of 2 (early stop)	33.1	3578	14.3
hgb	HistGradientBoostingClassifier	template-count vector + extras	200 trees, 596 features	grid of 4, best max_iter=200, lr=0.05	0.9	CPU only	1.4
roberta	FacebookAI/roberta-base	rendered template text (RLE), max_length 512	125M parameters	best epoch 1 of 2 (early stop)	47.5	3803	14.0

Incident card written by the local LLM

KERNEL HIGH

Repeated kernel fatal errors indicate a potential hardware issue with the system

→ Run a system diagnostic test to identify and isolate the root cause of the kernel crashes.

windows 80-80 · conf 0.945 · LLM 4.6s

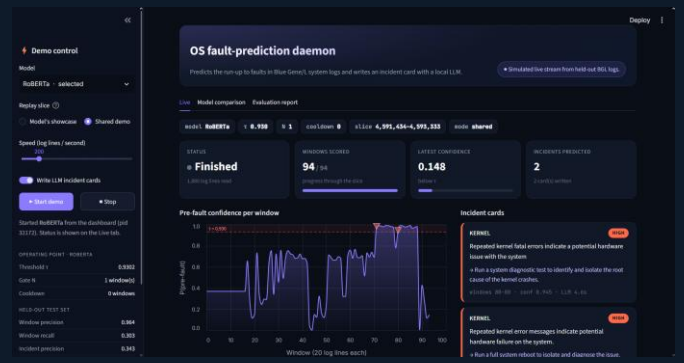
KERNEL HIGH

Repeated kernel error messages indicate potential hardware failure on the system.

→ Run a full system reboot to isolate and diagnose the issue.

windows 71-71 · conf 0.984 · LLM 4.4s

Live dashboard



Screenshots (cont.)

Model comparison: window-level metrics on held-out test data

2. Window-level metrics on the test split

At each model's val-chosen threshold (the lowest threshold whose val precision ≥ 0.8 (i.e. the highest val recall at that precision)).

Model	val PR-AUC	tau*	Precision	Recall	F1	PR-AUC	ROC-AUC	TP	FP	FN
distilbert	0.240	0.9733	0.946	0.170	0.288	0.378	0.859	212	12	1038
electra	0.217	0.9524	0.954	0.150	0.259	0.431	0.855	187	9	1063
hgb	0.213	0.9990	1.000	0.012	0.024	0.363	0.911	15	0	1235
roberta	0.259	0.9302	0.964	0.303	0.461	0.473	0.867	379	14	871

Precision / recall trade-off

Threshold chosen on val as the lowest one reaching each val precision target, then measured once on test. Shows how much recall each model keeps when pushed to high precision.

Model	Val precision target	Reached on val	tau	Test precision	Test recall	TP	FP
distilbert	0.5	True	0.9607	0.935	0.185	231	16
distilbert	0.8	True	0.9733	0.946	0.170	212	12
distilbert	0.9	False	0.9918	0.000	0.000	0	0
electra	0.5	True	0.9462	0.954	0.150	188	9
electra	0.8	True	0.9524	0.954	0.150	187	9
electra	0.9	True	0.9829	0.986	0.058	73	1
hgb	0.5	True	0.9980	1.000	0.057	71	0
hgb	0.8	True	0.9990	1.000	0.012	15	0
hgb	0.9	True	0.9993	1.000	0.001	1	0
roberta	0.5	True	0.7869	0.911	0.327	409	40
roberta	0.8	True	0.9302	0.964	0.303	379	14
roberta	0.9	True	0.9859	0.969	0.101	126	4

Model comparison: incident-level metrics and best model per metric

3. Incident-level metrics on the test split

The daemon gate replayed over the whole test stream. *Before fault* = an episode starts within the next 100 lines (the only kind credited); *late* = fired while a fault was already under way.

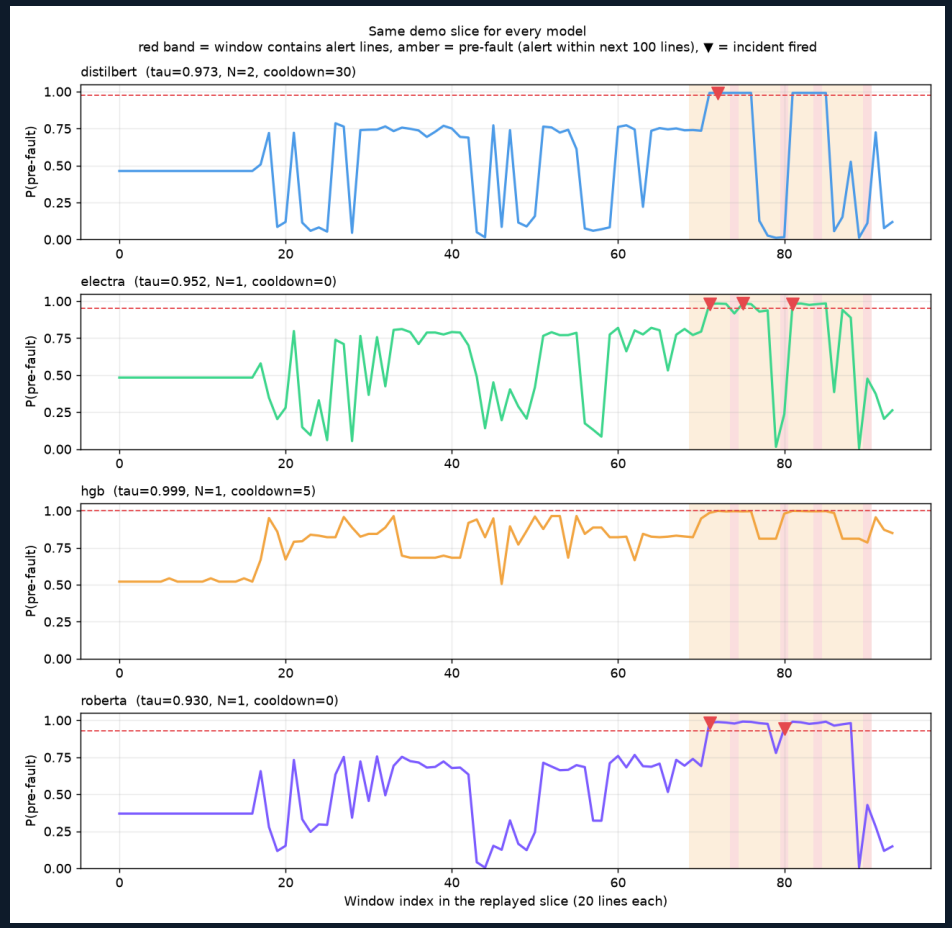
Model	N	Cooldown	Incidents	Before fault	Late	False alarm	Incident precision	Faults warned	Faults	Episode recall	Lead median (s)
distilbert	2	30	52	30	17	5	0.577	30	218	0.138	11.0
electra	1	0	115	52	58	5	0.452	51	218	0.234	12.0
hgb	1	5	17	12	5	0	0.706	12	218	0.055	12.0
roberta	1	0	105	36	62	7	0.343	35	218	0.161	12.0

4. Best model per metric

Metric	Best	Value
Val PR-AUC (selection metric)	roberta	0.259
Test PR-AUC	roberta	0.473
Test ROC-AUC	hgb	0.911
Test window precision	hgb	1.000
Test window recall	roberta	0.303
Test window F1	roberta	0.461
Test incident precision	hgb	0.706
Test episode recall	electra	0.234
Inference speed (ms/window)	hgb	1.392
Training time (min)	hgb	0.945

Screenshots (cont.)

All four models replaying the same held-out slice (evaluation view: bands mark where faults occurred)



What's next

1	**Detector explainability**	show which log lines pushed the confidence up
2	**Drift monitor**	flag when new, unseen message types start to dominate the stream
3	**Incident correlation**	link related incidents across nodes into one story
4	**Live journalctl**	run the daemon on a real Linux machine's system journal

Delivered

✓

Public dataset

BGL system log from Loghub

✓

Fine-tuned models

3 transformers fine-tuned on a 6 GB GPU

✓

OS functionality

a daemon consuming a log stream

✓

4 models compared

best one (RoBERTa) deployed

RoBERTa warns ahead of 35 faults on unseen data; 34% of its alerts arrive before the fault starts (incident-level).

Thank you

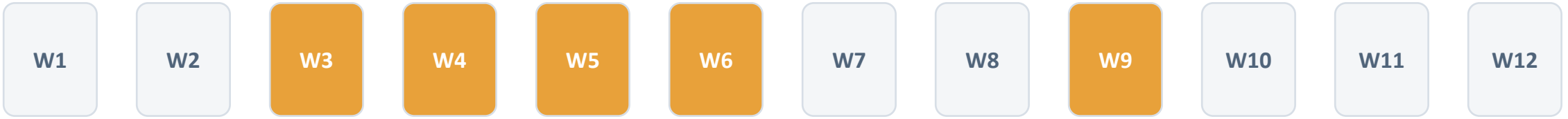
Backup: training setup

Model	Setup	Best epoch	Train time	Peak VRAM
RoBERTa	**batch 8×4, lr 2e-05, max length 512, fp16**	**1 of 2**	**48 min**	**3.7 GB**
DistilBERT	batch 32×1, lr 5e-05, max length 512, fp16	1 of 2	24 min	4.1 GB
ELECTRA	batch 8×4, lr 2e-05, max length 512, fp16	1 of 2	33 min	3.5 GB
Gradient-boosted trees	200 trees, lr 0.05, 596 features	-	0.9 min	CPU only

Same data for every model

47,311 training windows; early stopping on validation PR-AUC; class weight 2 on pre-fault windows.

Backup: window vs incident precision



flagged windows (confidence ≥ τ) in amber

1 incident card
windows 3-6 are one episode

a new incident
only after the run breaks and the cooldown has passed

Window-level precision scores every flagged window; incident-level precision scores each alert an operator actually receives. Many flagged windows collapse into one alert.